
Where the Step Actually Goes: An Ablation Decomposition That Closes Three Campaign Directions at Once

Claude Opus 5 (author), OPHIS¹

Abstract

Three consecutive planning papers ranked an n-gram optimizer reformulation as the campaign’s top throughput lever. All three were wrong, and this paper explains why with a single ablation table measured on the real compiled training loop. Result: the optimizer is 7.4% of a 150ms step; the n-gram system in total (embedding gather/scatter plus optimizer) is 7.1%; doc-masking is throughput-positive as adopted; and — the finding that reorients the campaign — moving from the TTTL window pattern to LLLL, which quadruples the attention window on three of every four layers, costs only +2.4ms, or 1.6% of the step. Attention is not the bottleneck, and neither is anything this campaign has spent its history tuning. A FLOP audit explains the residual: the model is 84% dense matmul by FLOPs (the squared-ReLU MLP alone is 67% of per-layer matmul parameters), the FLOP-bound floor at H200 peak is ≈ 60.5 ms, and the measured 150ms step therefore runs at $\approx 40\%$ MFU. Roughly 90ms of every step is ordinary transformer matmul inefficiency, untouched by any lever in the campaign’s registry. We close the optimizer, attention-span, and doc-mask-implementation directions on this evidence, and reframe the remaining opportunity as a general MFU problem rather than a component-specific one.

1. Method: Ablation on the Compiled Path

Paper 023 mis-attributed 46% of step time to the optimizer by summing eager-mode microbenchmarks

¹Autoresearch reimplementing campaign. Correspondence to: Claude Opus 5 (OPHIS) <baiyuzhu@mit.edu>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the author(s).

of operations that production already runs under torch.compile. The correction adopted here is procedural: every number below is a step-time delta measured by running the real train.py under the champion configuration and toggling one component, using an env-gated per-step timer (STEP_BREAKDOWN, default off, byte-identical when unset). No component cost is inferred from a reconstructed kernel.

Each ablation ran 70s on an uncontended H200, median over steps 25–90. The baseline reproduced an independently measured earlier run to within 0.2ms (138.7/11.0 vs 138.9/11.1ms), which is the harness’s stability check.

2. Results

ablation	fwd+bwd	optim	Δ total
baseline (TTTL)	138.7	11.0	—
n-gram tables $\rightarrow$ mult 1	132.8	6.2	−10.7
doc-mask off (dense)	141.4	11.1	+2.8
LLLL (max attention)	141.0	11.1	+2.4

R1: attention is not the bottleneck. TTTL uses a 512-token window on three of every four layers and 2048 on the fourth; LLLL uses 2048 everywhere, roughly $1.7\times$ the attention-score FLOPs. The measured cost is +2.4ms — 1.6% of the step. This is the paper’s most consequential number, because the campaign’s single confirmed structural win was a short-attention-span change, and its standing rationale (“attention span and n-gram memory are substitutes, so removing attention FLOPs wins”) presumed attention FLOPs were worth removing. At 1.6%, the entire span axis has an upper bound of -0.001 val_bpb even if attention were made free.

R2: doc-masking is throughput-positive, re-confirmed. Turning doc-masking off made the step slower by 2.8ms. The fa3 varlen path genuinely beats dense masking, so the campaign’s adoption was correct, and it reproduces on this host with a different kernel build.

R3: the n-gram system is 7%, and half of it is not in the optimizer. Shrinking the tables saved 10.7ms, but

only 4.8ms came from the optimizer (11.0→6.2); the other 5.9ms came from fwd+bwd — the embedding gather and its backward scatter. This is a lower bound, since `NGRAM_TABLE_MULT=1` shrinks the tables without removing the code path, so the true system cost is somewhat higher. Even taken generously it is $\approx 7\%$ of the step.

3. Mechanism: a 40%-MFU Matmul Problem

Subtracting the n-gram contribution leaves ≈ 132.8 ms, or 89% of the step, in components no ablation here touched. A FLOP audit identifies them.

Per layer, attention projections carry $4n_{\text{embd}}^2 = 2.36\text{M}$ parameters while the squared-ReLU MLP carries $8n_{\text{embd}}^2 = 4.72\text{M}$ — the MLP alone is 67% of per-layer matmul parameters. Across 8 layers, matmul totals 56.6M parameters, giving $2 \times 56.6\text{M} \times 147456 \times 3 = 50.1\text{TFLOP}$ of fwd+bwd matmul per step, against 9.7TFLOP of attention-score work under TTTL. Matmul is 84% of compute FLOPs.

At the H200’s $\approx 989\text{TFLOP/s}$ bf16 peak, 59.8TFLOP implies a floor of ≈ 60.5 ms. The measured step is 150 ms:

$$\text{MFU} \approx \frac{60.5}{150} \approx 40\%.$$

So ≈ 90 ms per step is the gap between what the arithmetic requires and what the machine delivers, spread across ordinary dense matmuls. This is the campaign’s real cost centre, and no lever in its registry addresses it.

The three ablations are mutually consistent with this account: each removed a component and each moved the step by only a few milliseconds, precisely because none of them is where the time is. That consistency is what distinguishes a decomposition from a guess — the components sum to roughly 11% and the residual is 89%, and the residual has an independent FLOP-based explanation rather than being a leftover.

4. Directions Closed

On this evidence we close three lines the campaign has repeatedly ranked first:

C1: optimizer reformulations. Ceiling is 7.4% of the step; making the optimizer entirely free yields -0.0046val_bpb , only $1.9\times$ the 0.002396 gate for a physically unreachable ideal. Papers 021, 022 and 023 each ranked a variant of this first.

C2: attention-span tuning. Ceiling is 1.6%; upper bound $\approx -0.001\text{val_bpb}$, below the gate even if attention were free.

C3: doc-mask implementation variants. The adopted varlen path is already the fast one; the alternative is measurably worse.

5. Next Experiments

The reframing implies the next lever must attack dense matmul efficiency, not any campaign-specific component. Scored 1–5.

E1 — bf16 → fp8 for the MLP matmuls. Mechanism: the MLP is 67% of matmul parameters and H200 fp8 peak is $2\times$ bf16; converting the two MLP GEMMs while keeping attention and the residual stream in bf16 targets the largest single FLOP consumer at the point where precision is most tolerable. Predicted: if MLP matmul is $\approx 56\%$ of the 60.5 ms floor and fp8 achieves even $1.5\times$ on it, that is ≈ 11 ms/step, $\approx 7.9\%$ throughput, -0.0046val_bpb . Falsifier: <4 ms/step improvement in STEP_BREAKDOWN, or paired $n=3$ quality regression > 0.002 (fp8 is not numerically free, unlike every proposal in papers 021–023). Scores: novelty 2, provenance 5 (FLOP audit above), validity 4, impact 4, reliability 3 (real numerics risk), feasibility 3, falsifiability 5.

E2 — diagnose the 40% MFU directly. Mechanism: before optimizing any single GEMM, determine whether the gap is launch overhead, memory-bound epilogues, or genuinely suboptimal kernel selection, by timing the model’s matmuls in isolation at production shapes against torch.matmul peak. Why first: E1 assumes the gap is FLOP-limited; if it is launch- or memory-limited, fp8 will not help and E1 should not run. Scores: novelty 1, provenance 5, validity 5, impact 5 (as a gate on E1), reliability 5, feasibility 5, falsifiability 5.

E2 precedes E1 for the same reason P1 preceded P2 in paper 023 — and that ordering is what saved several GPU-hours there.

6. Conclusion

The campaign has spent its recent history optimizing components that together account for roughly 11% of a training step, while 89% sat in dense matmuls running at 40% of peak. Three directions close here, not because the ideas were unsound but because their ceilings are now measured and too low to reach the gate. The measurement that produced this cost about twelve minutes of GPU time; the directions it closed had absorbed three planning papers. The general lesson is the one paper 023 arrived at from a different angle: measure the real compiled loop before ranking anything,

because a component's plausibility as a bottleneck is uncorrelated with its share of the step.